Experiment 8: Jenkins – Continuous Integration using GitHub (Ubuntu)

Ubuntu practical

Step-by-step from zero

In this practical, we will create a GitHub repository, install Jenkins on Ubuntu, connect Jenkins to GitHub and automatically execute a build whenever we push code changes.

1. Software and requirements

- Ubuntu 24.04 (recommended for these instructions).
- Internet connection.
- GitHub account.
- Java 21.
- Jenkins LTS.
- Git.

What is Jenkins?

Jenkins is an open-source automation server used to automatically build, test and deploy software projects.

Practical workflow:

GitHub repository

Project files + Jenkinsfile

Jenkins checks GitHub

Pipeline runs Build and Test stages

Build SUCCESS

2. Install Git on Ubuntu

Open Ubuntu Terminal using Ctrl + Alt + T.

Update the package list:

sudo apt update

Install Git:

sudo apt install git -y

Check Git version:

git --version

Expected output (version may differ):

git version 2.x.x

Configure your GitHub username and email:

git config --global user.name "Your Name"

git config --global user.email "your-email@gmail.com"

Replace these with your GitHub account details.

3. Create a GitHub repository

1. Open GitHub.
2. Sign in.
3. Click + → New repository.
4. Enter repository name:

Jenkins-GitHub-CI

5. Select Public.
6. Select Add a README file.
7. Click Create repository.

Your repository will look like:

https://github.com/YOUR-USERNAME/Jenkins-GitHub-CI

Replace YOUR-USERNAME with your actual GitHub username.

4. Create project folder in Ubuntu

In Terminal, enter:

mkdir Jenkins-GitHub-CI

Go inside the folder:

cd Jenkins-GitHub-CI

Create a Python file:

nano hello.py

Write:

print("Hello! Jenkins CI Pipeline")print("Build executed successfully")

Save the file:

- Press Ctrl + O.
- Press Enter.
- Press Ctrl + X.

5. Create Jenkinsfile

In the same folder, enter:

nano Jenkinsfile

Write this complete pipeline:

```
pipeline {
    agent any

    stages {

        stage('Build') {
            steps {
                echo 'Building the project...'
            }
        }

        stage('Test') {
            steps {
                echo 'Testing the project...'
                sh 'test -f hello.py'
```

```
            echo 'Python file found'
        }
    }

    stage('Success') {
        steps {
            echo 'Jenkins CI Pipeline completed successfully!'
        }
    }
  }
}
```

Save using Ctrl + O, Enter, and Ctrl + X.

Understand the Jenkinsfile:

Command	Meaning
pipeline	Defines the pipeline.
agent any	Runs on any available agent.
stages	Defines different stages.
stage('Build')	Build stage.
stage('Test')	Test stage.
sh	Executes a Linux shell command.
echo	Displays a message.

Here, test -f hello.py checks whether the Python file exists.

6. Upload project files to GitHub

Initialize Git:

```
git init
```

Set the branch name:

git branch -M main

Add your GitHub repository as remote:

git remote add origin https://github.com/YOUR-USERNAME/Jenkins-GitHub-CI.git

Add the project files:

git add .

Commit:

git commit -m "Add Jenkins pipeline project"

Push to GitHub:

git push -u origin main

If GitHub asks for authentication, use your GitHub username and a Personal Access Token (PAT) instead of your account password, or complete the authentication flow offered by Git.

Open your GitHub repository and verify that these two files are visible:

Jenkins-GitHub-CI/

hello.py

Jenkinsfile

Project uploaded to GitHub

7. Install Java 21

Jenkins requires a supported Java runtime. We will install OpenJDK 21.

Jenkins+1

In Ubuntu Terminal:

sudo apt update

Install Java:

sudo apt install fontconfig openjdk-21-jre -y

Check Java:

java -version

Expected output:

openjdk version "21..."

OpenJDK Runtime Environment

Java installed

8. Install Jenkins on Ubuntu

We will install the stable Jenkins LTS version using the official Jenkins repository.

Jenkins+1

Step 1: Add Jenkins repository key

sudo mkdir -p /etc/apt/keyrings

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \

https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

Step 2: Add Jenkins repository

Copy this complete command:

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

Step 3: Update package list

sudo apt update

Step 4: Install Jenkins

sudo apt install jenkins -y

Wait until installation finishes.

9. Start Jenkins service

Start Jenkins:

sudo systemctl start jenkins

Enable Jenkins to start automatically when Ubuntu starts:

sudo systemctl enable jenkins

Check its status:

sudo systemctl status jenkins

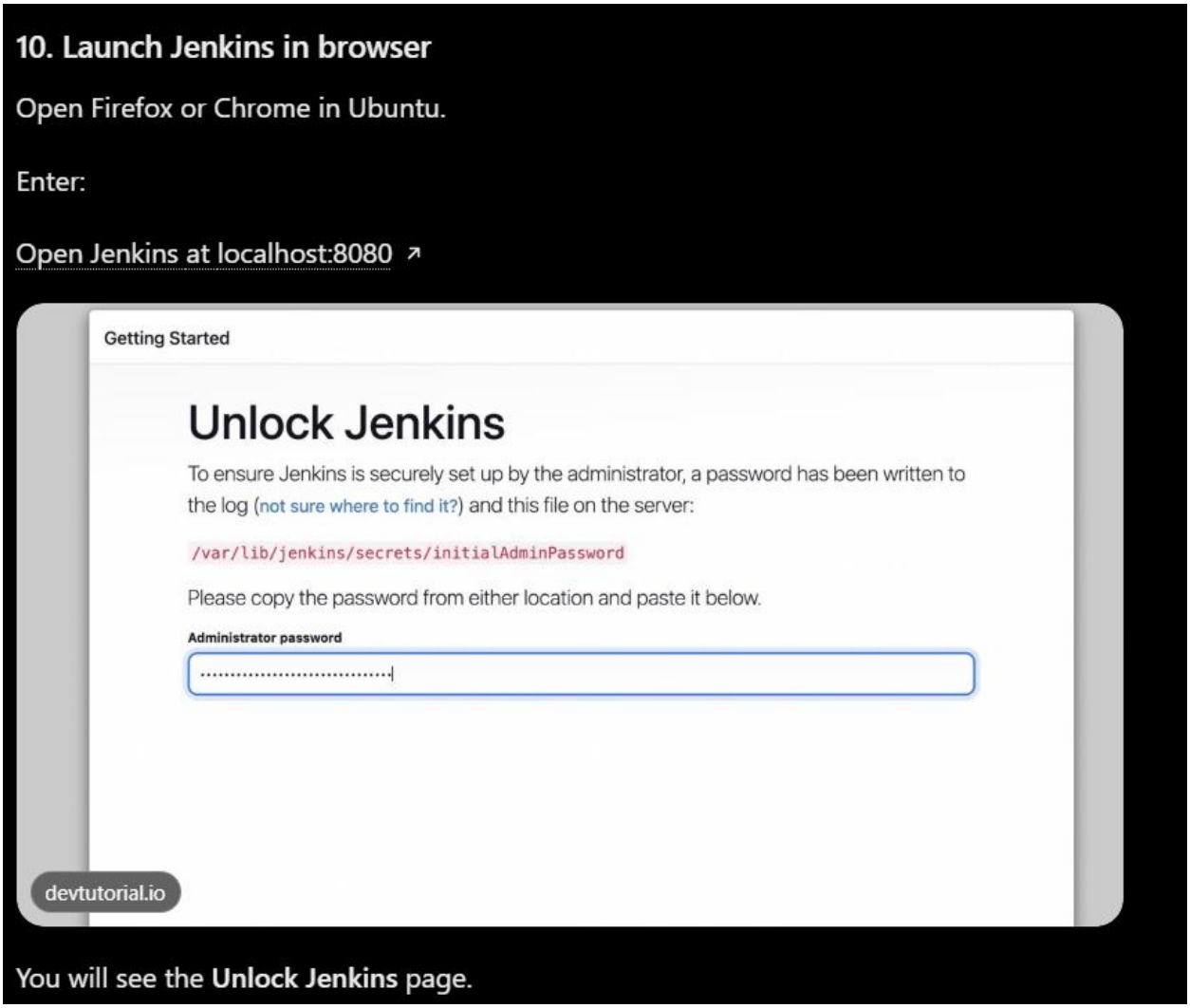
Expected output should include:

Active: active (running)

Jenkins service running

Press q to exit the status screen.

10. Launch Jenkins in browser



Open Firefox or Chrome in Ubuntu.

Enter:

Open Jenkins at localhost:8080

How to Install and configure Jenkins on Ubuntu 24.10 - Devtutorial

devtutorial.io

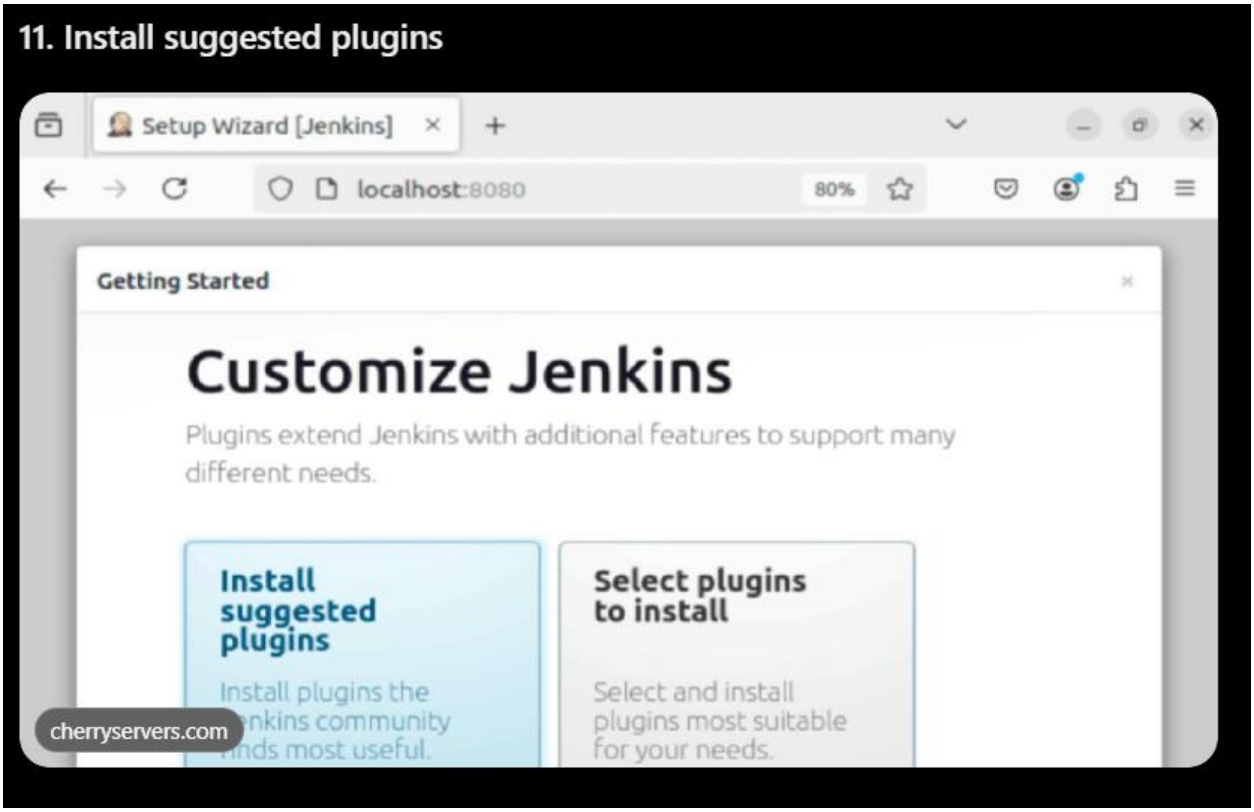
You will see the Unlock Jenkins page.

Find the initial password by entering this command in Terminal:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

1. Copy the displayed password.
2. Paste it into the Jenkins browser page.
3. Click Continue.

11. Install suggested plugins



How to Install Jenkins on Ubuntu 24.04: Step-by-Step | Cherry Servers

cherryservers.com

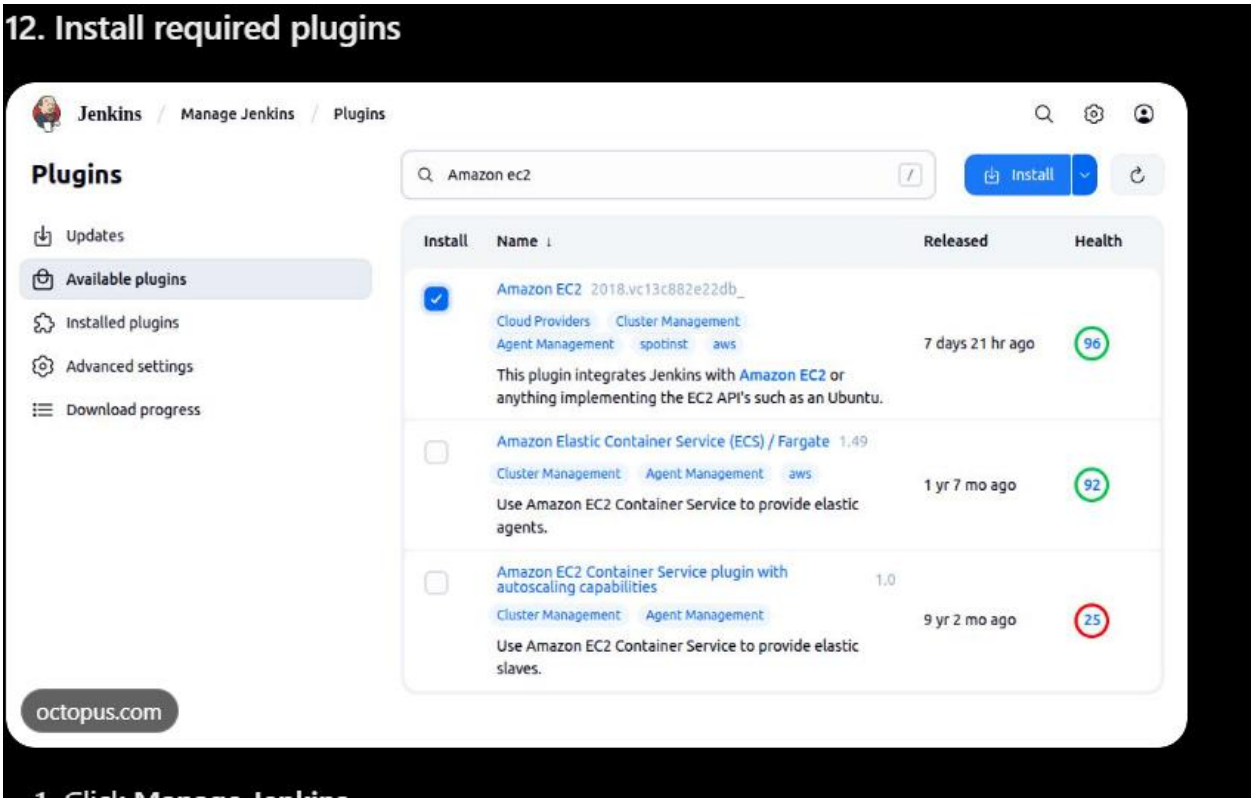
1. Click Install suggested plugins.
2. Wait until the installation completes.
3. Create your administrator account:

- Username: choose your username.
- Password: create your password.
- Full name: enter your name.
- Email: enter your email.

4. Click Save and Continue.
5. Click Save and Finish.
6. Click Start using Jenkins.

Now Jenkins dashboard should open.

12. Install required plugins



[Deploying Jenkins On AWS: Tutorial And Best Practices | Octopus Deploy

octopus.com]

1. Click Manage Jenkins.
2. Open Plugins.
3. Search for these plugins:

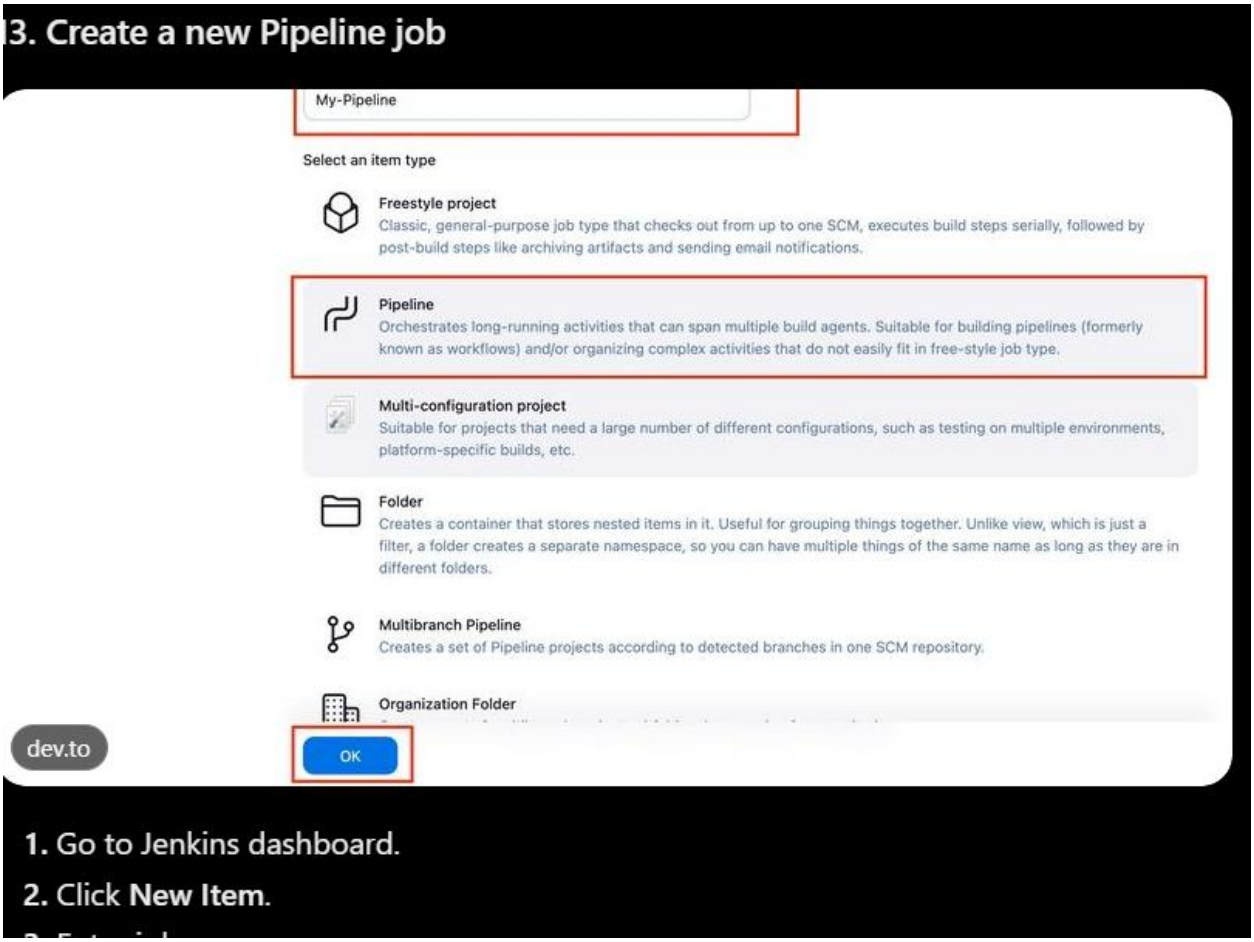
Plugin	Purpose
Git	Connect Jenkins to Git repositories.
GitHub	GitHub integration.
Pipeline	Create and run Pipeline jobs.

4. Install any that are not already installed.
5. Restart Jenkins if requested.

Some plugins may already be installed through suggested plugins. You do not need to install them twice.

Jenkins plugin

13. Create a new Pipeline job



[How to create a Jenkins Pipeline job - DEV Community]

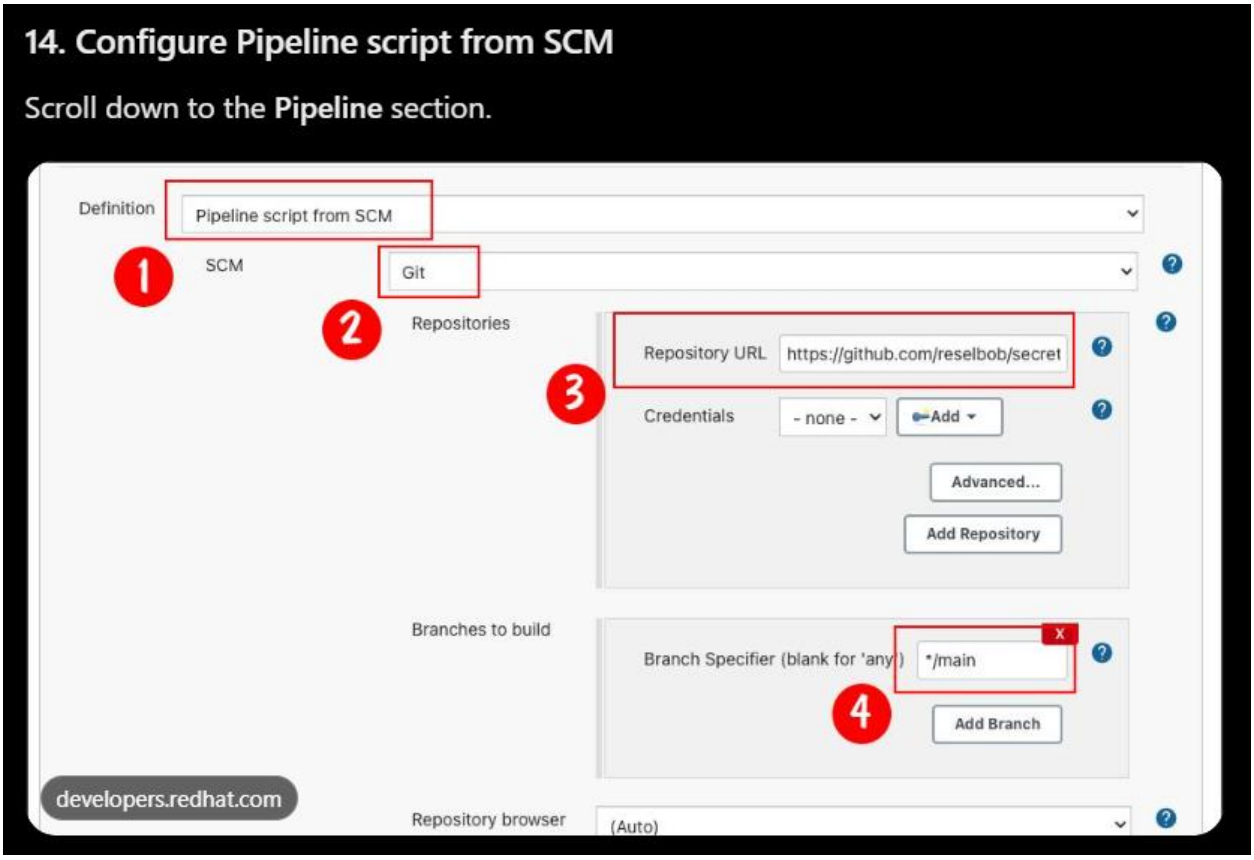
dev.to

1. Go to Jenkins dashboard.
2. Click New Item.
3. Enter job name:

MyFirstPipeline

4. Select Pipeline.
5. Click OK.

14. Configure Pipeline script from SCM



Scroll down to the Pipeline section.

Automating CI/CD and GitOps with Jenkins | Red Hat Developer

developers.redhat.com

Set the following details:

Option	Value
Definition	Pipeline script from SCM
SCM	Git
Repository URL	https://github.com/YOUR-USERNAME/Jenkins-GitHub-CI.git
Credentials	None (for public repository)
Branch Specifier	*/main
Script Path	Jenkinsfile

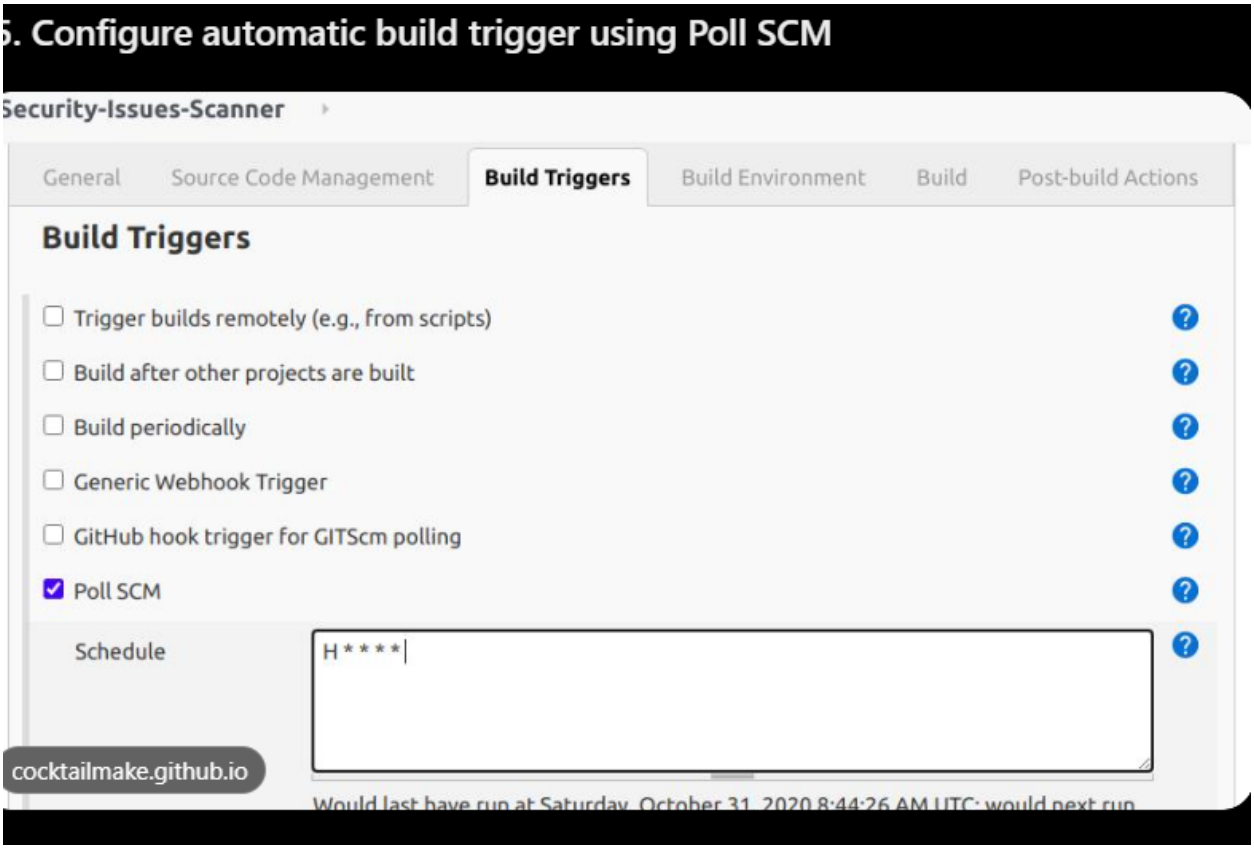
Replace YOUR-USERNAME with your GitHub username.

Click Save.

Jenkins will fetch your repository and read the Jenkinsfile.

Jenkins+1

15. Configure automatic build trigger using Poll SCM



[Find Security Issues With Bandit | RICHKA Developer Blog]

[cocktailmake.github.io]

1. Open MyFirstPipeline.
2. Click Configure.
3. Find the Build Triggers section.
4. Tick Poll SCM.
5. In the Schedule box, enter:

H/1 * * * *

Meaning: Jenkins checks the GitHub repository approximately every minute for changes.

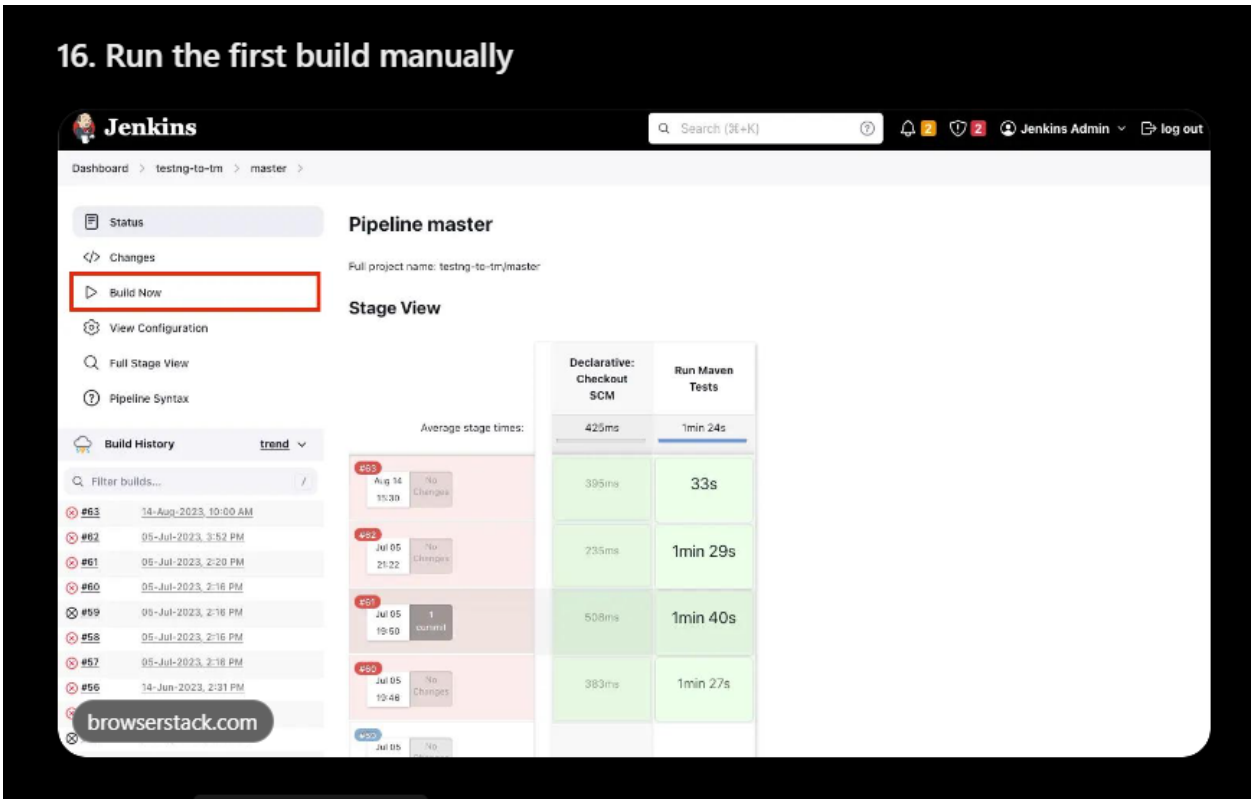
6. Scroll down and click Save.

Why Poll SCM?

It checks the GitHub repository periodically. When a new commit is detected, Jenkins automatically starts a build.

We are using Poll SCM instead of a GitHub webhook because Jenkins is running on your local computer at localhost:8080. GitHub cannot directly reach your local Jenkins server without additional public network configuration.

16. Run the first build manually



[Integrate BrowserStack Test Reporting & Analytics with Test Management | BrowserStack Docs](browserstack.com)

1. Open MyFirstPipeline.
2. Click Build Now.
3. Wait for the build to complete.
4. Look at Build History.
5. Click build number #1.
6. Click Console Output.

Expected output:

Building the project...

Testing the project...

Python file found

Jenkins CI Pipeline completed successfully!

Finished: SUCCESS

First build successful

17. Push code changes to GitHub

Now we will check automatic build triggering.

Open Ubuntu Terminal and go inside your project folder:

cd ~/Jenkins-GitHub-CI

Open the Python file:

nano hello.py

Add one more line:

print("This is my second Jenkins build")

Your complete file will be:

print("Hello! Jenkins CI Pipeline")print("Build executed successfully")print("This is my second Jenkins build")

Save:

- Ctrl + O
- Enter
- Ctrl + X

Check changes:

git status

Add changes:

git add hello.py

Commit changes:

git commit -m "Update hello Python file"

Push to GitHub:

git push origin main

18. Verify automatic build

DAY 37/90=> Jenkins Pipeline. What is a Jenkins Pipeline? | by Faisal Kuzhan | Medium

medium.com

1. Open Jenkins:

Jenkins dashboard

2. Open MyFirstPipeline.
3. Wait approximately 1–2 minutes.
4. Check Build History.
5. A new build, such as #2, should appear.
6. Click #2.
7. Open Console Output.

Expected output:

Building the project...

Testing the project...

Python file found

Jenkins CI Pipeline completed successfully!

Finished: SUCCESS

Automatic build verified

This completes the practical: Jenkins detects your GitHub changes through Poll SCM and automatically executes the pipeline.

19. Troubleshooting

Problem	Solution
Jenkins page not opening	Run sudo systemctl status jenkins.
Java not installed	Run java -version and reinstall Java 21 if necessary.
Jenkins not running	Run sudo systemctl start jenkins.
Port 8080 already in use	Check sudo ss -ltnp | grep 8080.
Cannot find password	Run sudo cat /var/lib/jenkins/secrets/initialAdminPassword.
Git repository error	Check the repository URL and its visibility.
Build fails	Open Console Output and inspect the error.
Automatic build not starting	Check Poll SCM schedule and confirm that you pushed a new commit.

20. Important viva questions

Question	Answer
What is Jenkins?	An open-source automation server.
What is CI?	Continuous Integration.
What is a Pipeline?	A sequence of automated stages.
What is Jenkinsfile?	A file containing pipeline instructions.
What is a plugin?	An extension that adds functionality to Jenkins.
What is Poll SCM?	Periodically checks a repository for code changes.
What is a webhook?	A notification sent when an event occurs in GitHub.
What is Build Now?	Manually starts a Jenkins build.
What is Console Output?	Displays build execution logs.

What is SCM?	Source Code Management.
What does SUCCESS mean?	The build completed successfully.
What is localhost?	The local computer on which Jenkins is running.
Which port does Jenkins use by default?	8080.

21. Final practical checklist

Experiment 8 progress

0/14

Created GitHub repository

Created hello.py

Created Jenkinsfile

Uploaded project files to GitHub

Installed Java 21

Installed Jenkins on Ubuntu

Started Jenkins at localhost:8080

Installed required plugins

Created Pipeline job

Configured Pipeline script from SCM

Configured Poll SCM

Executed first build successfully

Pushed code changes to GitHub

Verified automatic second build

Official references

- Jenkins installation on Linux
- Jenkins Pipeline documentation
- Getting started with Jenkins Pipeline

Important: For this Ubuntu practical, remember these three things:

1. GitHub stores the project.
2. Jenkins automatically runs the pipeline.
3. Jenkinsfile contains the build and test instructions.

Exam tip: Do not skip the first manual build. Run it once before testing automatic triggering so Jenkins has an initial build and repository history to compare against.

Experiment 9: Docker – Containerization of an AI Model (Ubuntu)

Complete practical

1. Aim

To containerize an AI model using Docker by creating a Docker image, running a Docker container, testing the application through a browser, and stopping the container after testing.

2. Software and tools required

- Ubuntu Operating System
- Docker
- Python
- Flask
- Scikit-learn
- Web Browser

3. Theory

Docker: Docker is a platform used to develop, package, and run applications inside containers.

Docker Image: A Docker image is a package containing application code, libraries, and dependencies.

Docker Container: A Docker container is a running instance of a Docker image.

Dockerfile: A Dockerfile contains instructions to build a Docker image.

Flask: Flask is a Python framework used to create web applications and APIs.

4. Procedure / Implementation

Step 1: Check Docker installation

Open Ubuntu Terminal using Ctrl + Alt + T.

Check Docker version:

docker --version

Check Docker service:

sudo systemctl status docker

If Docker is installed but not running:

sudo systemctl start docker

Step 2: Create a project directory

mkdir AI-Docker-Project

cd AI-Docker-Project

Check the current directory:

pwd

Step 3: Create the AI application (app.py)

nano app.py

Write the following Python code:

from flask import Flask, jsonifyfrom sklearn.datasets import load_irisfrom sklearn.tree import DecisionTreeClassifierapp = Flask(__name__)# Load Iris datasetiris = load_iris()# Train AI modelmodel = DecisionTreeClassifier()model.fit(iris.data, iris.target)# Home page@app.route('/')def home(): return "AI Model is Running Inside Docker!"# Prediction API@app.route('/predict')def predict(): sample = [[5.1, 3.5, 1.4, 0.2]] prediction = model.predict(sample) result = iris.target_names[prediction[0]] return jsonify({ "prediction": result, "message": "AI prediction successful" })# Run Flask applicationif __name__ == '__main__': app.run(host='0.0.0.0', port=5000)

Save the file using Ctrl + O, press Enter, and then Ctrl + X.

Explanation:

- Loads the Iris dataset.
- Trains a Decision Tree classification model.
- Creates a Flask web application.
- Provides a prediction API at /predict.
- Runs the application on port 5000.

Step 4: Create requirements.txt

nano requirements.txt

Write:

flask

scikit-learn

Save the file.

This file contains the Python libraries required to execute the AI application.

Step 5: Create a Dockerfile

nano Dockerfile

Write:

```
# Use Python base image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application file
COPY app.py .

# Expose port 5000
EXPOSE 5000

# Run Flask application
CMD ["python", "app.py"]
```

Save the file.

Explanation of Dockerfile instructions:

Instruction	Description
FROM	Selects the Python base image
WORKDIR	Sets the working directory
COPY	Copies files into the image
RUN	Executes commands during image building
EXPOSE	Documents the application port
CMD	Specifies the command to run the application

Step 6: Verify project files

Run:

ls

Expected output:

app.py

requirements.txt

Dockerfile

Step 7: Build the Docker image

Run:

sudo docker build -t ai-model-app .

Explanation:

- docker build creates a Docker image.
- -t ai-model-app assigns a name to the image.
- . specifies the current directory as the build context.

Check the created image:

sudo docker images

Expected output will contain:

REPOSITORY TAG IMAGE ID CREATED

ai-model-app latest xxxxxxxxx ...

Step 8: Run the Docker container

Execute:

sudo docker run -d -p 5000:5000 --name ai-container ai-model-app

Explanation:

Command	Purpose
-d	Runs container in background
-p 5000:5000	Maps host port 5000 to container port 5000
--name	Assigns a container name
ai-model-app	Specifies the Docker image

Step 9: Verify the running container

Run:

sudo docker ps

Expected output:

CONTAINER ID IMAGE STATUS PORTS

xxxxxxxx ai-model-app Up ... 0.0.0.0:5000->5000/tcp

The status should show Up, indicating that the container is running.

Step 10: Test the application in a browser

Open your web browser.

Home page:

Open AI application

Expected output:

AI Model is Running Inside Docker!

Test the prediction API:

Open AI prediction API

Expected JSON output:

```
{
   "prediction": "setosa",
   "message": "AI prediction successful"
}
```

The AI model predicts the Iris flower category using the input features.

You can also test using Ubuntu Terminal:

curl http://localhost:5000

curl http://localhost:5000/predict

Step 11: View container logs

Run:

sudo docker logs ai-container

This displays the application logs generated by the running container.

Step 12: Stop the Docker container

After testing, execute:

sudo docker stop ai-container

Verify running containers:

sudo docker ps

The stopped container should not appear in the running-container list.

To view all containers, including stopped ones:

sudo docker ps -a

5. Final project structure

AI-Docker-Project/

|

├── app.py

│

├── requirements.txt

│

└── Dockerfile

6. Important Docker commands

Command	Purpose
docker --version	Checks Docker version
docker images	Lists Docker images
docker build -t ai-model-app .	Builds Docker image
docker run -d -p 5000:5000 --name ai-container ai-model-app	Runs container
docker ps	Shows running containers
docker ps -a	Shows all containers
docker logs ai-container	Displays container logs
docker stop ai-container	Stops container

7. Result

Successfully created an AI model application using Python and Flask, prepared a Dockerfile, built a Docker image, and executed the application inside a Docker container. The application was tested through localhost:5000, its running status was verified, and the container was stopped successfully.

8. Viva questions and answers

Q1. What is Docker?

Docker is a platform used to package and run applications inside containers.

Q2. What is a Docker image?

A Docker image is a read-only template used to create containers.

Q3. What is a Docker container?

A Docker container is an instance of a Docker image.

Q4. What is a Dockerfile?

A Dockerfile is a text file containing instructions to build a Docker image.

Q5. What is the purpose of requirements.txt?

It contains the Python libraries required by the application.

Q6. What is the difference between Docker image and Docker container?

An image is the template, whereas a container is the running instance of that image.

Q7. What is the purpose of port mapping?

Port mapping connects a port on the host machine to a port inside the container.

Q8. What is the purpose of docker build?

It creates a Docker image using instructions from the Dockerfile.

Q9. What is the purpose of docker run?

It creates and starts a Docker container.

Q10. What is the purpose of docker stop?

It stops a running Docker container.

Q11. What is Flask?

Flask is a lightweight Python web framework used to develop web applications and APIs.

Q12. What is Scikit-learn?

Scikit-learn is a Python machine-learning library used for classification, regression, and clustering.

Experiment 9 complete.

Important: Run these commands in Ubuntu, not Windows PowerShell. If Docker is installed only on Windows, first configure Docker Desktop integration with your Ubuntu/WSL environment or install Docker Engine in Ubuntu.

Experiment 10: Docker Compose – Multi-Container Web-Based AI Application (Ubuntu)

1. Aim

To use Docker Compose to create, manage, and orchestrate multiple containers for a web-based AI application with separate services for:

1. Frontend: Displays the web application.
2. AI Model: Performs machine-learning predictions.
3. Database: Stores prediction records.

2. Software requirements

- Ubuntu
- Docker
- Docker Compose
- Python
- Flask
- Scikit-learn
- SQLite database

3. Theory

Docker Compose is a tool used to define and run multiple Docker containers using a single configuration file called docker-compose.yml.

In this experiment, three services communicate with each other:

Frontend Container

Web page – Port 5000

AI Model Container

Flask API – Port 5001

Database Container

Stores prediction history

We will use Flask for the frontend and AI API, a Scikit-learn Iris classifier for predictions, and a separate SQLite database container to store prediction records.

4. Procedure

Step 1: Check Docker and Docker Compose

Open Ubuntu Terminal using Ctrl + Alt + T.

Check Docker:

docker --version

Check Docker Compose:

docker compose version

Check whether Docker is running:

sudo systemctl status docker

If Docker is installed but stopped:

sudo systemctl start docker

If these commands are unavailable in Ubuntu, configure Docker for Ubuntu before continuing. Docker installed on Windows alone does not guarantee it is available inside Ubuntu.

Step 2: Create the project directory

mkdir Docker-Compose-AI

cd Docker-Compose-AI

Create three service directories:

mkdir frontend ai-model database

The project will contain three separate services.

Step 3: Create the AI model service

Create the AI model Python file:

nano ai-model/app.py

Paste:

from flask import Flask, request, jsonifyfrom sklearn.datasets import load_irisfrom sklearn.tree import DecisionTreeClassifierimport psycopg2import osimport timeapp = Flask(__name__)# Connect to PostgreSQL databasedef get_connection():   return psycopg2.connect(host=os.getenv("DB_HOST", "database"),   database=os.getenv("POSTGRES_DB", "aidb"),

user=os.getenv("POSTGRES_USER", "admin"), password=os.getenv("POSTGRES_PASSWORD", "admin123"))# Create prediction history tabledef create_table(): for attempt in range(15): try: connection = get_connection() cursor = connection.cursor() cursor.execute(""" CREATE TABLE IF NOT EXISTS predictions (id SERIAL PRIMARY KEY, input_data TEXT, prediction TEXT) """) connection.commit() cursor.close() connection.close()

Save with Ctrl + O, Enter, and Ctrl + X.

Create the AI service requirements file:

nano ai-model/requirements.txt

Paste:

flask

scikit-learn

psycopg2-binary

Save the file.

Create its Dockerfile:

nano ai-model/Dockerfile

Paste:

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5001

CMD ["python", "app.py"]

Step 4: Create the frontend service

Create the frontend application:

nano frontend/app.py

Paste:

from flask import Flask, request, render_template_stringimport requestsimport osapp = Flask(__name__)AI_API_URL = os.getenv("AI_API_URL", "http://ai-model:5001")HTML = """<!DOCTYPE html><html><head> <title>AI Prediction Application</title> <style> body { font-family: Arial; background: #f0f2f5; text-align: center; padding: 30px; } .container { background: white; padding: 25px; max-width: 450px; margin: auto; border-radius: 10px; } input { width: 90%; padding: 10px; margin: 7px;

Save the file.

Create frontend requirements:

nano frontend/requirements.txt

Paste:

flask

requests

Save the file.

Create frontend Dockerfile:

nano frontend/Dockerfile

Paste:

FROM python:3.11-slim

WORKDIR /app

```
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Save the file.

Step 5: Create the database service

The database will use PostgreSQL. Docker Compose will run it in its own container.

We do not need to create a separate Python program for the database because the PostgreSQL Docker image provides the database service.

Step 6: Create the Docker Compose file

Go to the main project directory:

```
cd ~/Docker-Compose-AI
```

Create the Compose configuration:

```
nano docker-compose.yml
```

Paste:

```
services:

  frontend:
    build: ./frontend
    container_name: ai-frontend
    ports:
```

```
    - "5000:5000"
  environment:
    AI_API_URL: http://ai-model:5001
  depends_on:
    - ai-model

ai-model:
  build: ./ai-model
  container_name: ai-model
  ports:
    - "5001:5001"
  environment:
    DB_HOST: database
    POSTGRES_DB: aidb
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: admin123
  depends_on:
    database:
      condition: service_healthy

database:
  image: postgres:16
  container_name: ai-database
  environment:
    POSTGRES_DB: aidb
    POSTGRES_USER: admin
```

```
    POSTGRES_PASSWORD: admin123
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U admin -d aidb"]
    interval: 5s
    timeout: 5s
    retries: 10

volumes:
  postgres_data:
```

Save the file.

Explanation of important Docker Compose instructions:

Instruction	**Purpose**
services	Defines multiple containers
frontend	Creates the frontend service
ai-model	Creates the AI prediction service
database	Creates the PostgreSQL service
build	Builds an image using a Dockerfile
image	Uses an existing Docker image
ports	Maps host and container ports
environment	Sets environment variables
depends_on	Defines service startup dependencies
volumes	Preserves database data

healthcheck Checks database readiness

The services communicate using their Compose service names. For example, the frontend connects to ai-model:5001, and the AI service connects to database.

Step 7: Verify project structure

Run:

tree

If tree is not installed:

sudo apt install tree -y

Expected project structure:

```
Docker-Compose-AI/
|
 ├── docker-compose.yml
|
 ├── frontend/
|    ├── app.py
|    ├── requirements.txt
|    └── Dockerfile
|
└── ai-model/
     ├── app.py
     ├── requirements.txt
     └── Dockerfile
```

Step 8: Build and start all containers

Run this command from the main project directory:

sudo docker compose up -d --build

Explanation:

- docker compose manages multiple containers.
- up creates and starts the services.
- -d runs them in the background.
- --build builds the service images before starting them.

Wait for the services to start.

Step 9: Verify all containers

Run:

sudo docker compose ps

Expected output should show these three services:

Container	Service	Expected status
ai-frontend	Frontend	Running
ai-model	AI Model	Running
ai-database	Database	Running / Healthy

Check logs if a service is not running:

sudo docker compose logs

To view logs for a particular service:

sudo docker compose logs ai-model

Step 10: Test the web application

Open your browser and visit:

Open AI web application

You should see the AI Iris Flower Prediction page.

The default values are:

Input	Value
Sepal Length	5.1
Sepal Width	3.5

Petal Length 1.4

Petal Width 0.2

Click Predict.

Expected result:

Prediction: setosa

The prediction should also appear in the Prediction History section because the AI service stores it in PostgreSQL.

Step 11: Test the AI API

Open Ubuntu Terminal and execute:

curl http://localhost:5001/health

Expected output:

{"status":"AI service is running"}

Test the prediction API:

```
curl -X POST http://localhost:5001/predict \
-H "Content-Type: application/json" \
-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
```

Expected output:

```
{
  "message": "Prediction saved successfully",
  "prediction": "setosa"
}
```

Step 12: Verify database records

Run:

sudo docker compose exec database psql -U admin -d aidb

Inside the PostgreSQL prompt, execute:

SELECT * FROM predictions;

You should see the stored prediction records.

Exit PostgreSQL:

\q

Step 13: Stop all containers

After testing, return to the Ubuntu Terminal and run:

sudo docker compose down

This stops and removes the containers and Compose network. The named database volume remains, so saved prediction history is preserved.

To check containers:

sudo docker ps

The three containers should no longer appear in the running-container list.

To stop containers and also remove saved database data, use sudo docker compose down -v only if you intentionally want to delete that data.

Step 3: Create the AI model service

Create the AI model Python file:

nano ai-model/app.py

Paste this code:

from flask import Flask, request, jsonify connection = get_connection() cursor = connection.cursor() cursor.execute("INSERT INTO predictions (input_data, prediction) VALUES (%s, %s)", (str(values), result)) connection.commit() cursor.close() connection.close() return jsonify({ "prediction": result, "message": "Prediction saved successfully" }) except (KeyError, TypeError, ValueError): return jsonify({"error": "Enter four valid numeric values"}), 400# Display prediction history@app.route('/history')def history(): connection = get_connection() cursor = connection.cursor() cursor.execute("SELECT id, input_data, prediction FROM predictions ORDER BY id DESC") records = cursor.fetchall() cursor.close() connection.close() return jsonify([{ "id": row[0], "input": row[1], "prediction": row[2] } for row in records])@app.route('/health')def health(): return jsonify({"status": "AI service is running"})if __name__ == '__main__': create_table() app.run(host='0.0.0.0', port=5001)

Save the file using Ctrl + O, Enter, and Ctrl + X.

Create the requirements file:

nano ai-model/requirements.txt

Paste:

flask

scikit-learn

psycopg2-binary

Save the file.

Create the AI model Dockerfile:

nano ai-model/Dockerfile

Paste:

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5001

CMD ["python", "app.py"]

Save the file.

Step 4: Create the frontend service

Create the frontend Python application:

nano frontend/app.py

Paste:

from flask import Flask, request, render_template_stringimport requestsimport osapp = Flask(__name__)AI_API_URL = os.getenv("AI_API_URL", "http://ai-model:5001")HTML = """<!DOCTYPE html><html><head> <title>AI Prediction Application</title> <style> body { font-family: Arial; background: #f0f2f5; text-align: center; padding: 30px; } .container { background: white; padding: 25px; max-width: 450px; margin: auto; border-radius: 10px; } input { width: 90%; padding: 10px; margin: 7px; }

Save the file.

Create frontend requirements:

nano frontend/requirements.txt

Paste:

flask

requests

Save the file.

Create frontend Dockerfile:

nano frontend/Dockerfile

Paste:

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

Save the file.

Step 5: Configure the database service

We will use PostgreSQL as a separate database container.

No separate Python file is required for this service because Docker will download and run the official PostgreSQL image.

The AI model service will save its predictions in this database.

Step 6: Create the Docker Compose file

Return to the main project directory:

cd ~/Docker-Compose-AI

Create the Compose file:

nano docker-compose.yml

Paste:

services:

```
  frontend:
    build: ./frontend
    container_name: ai-frontend
    ports:
      - "5000:5000"
    environment:
      AI_API_URL: http://ai-model:5001
    depends_on:
```

```
    - ai-model

ai-model:
  build: ./ai-model
  container_name: ai-model
  ports:
    - "5001:5001"
  environment:
    DB_HOST: database
    POSTGRES_DB: aidb
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: admin123
  depends_on:
    database:
      condition: service_healthy

database:
  image: postgres:16
  container_name: ai-database
  environment:
    POSTGRES_DB: aidb
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: admin123
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
```

```
    test: ["CMD-SHELL", "pg_isready -U admin -d aidb"]
    interval: 5s
    timeout: 5s
    retries: 10

volumes:
  postgres_data:
```

Save the file.

Explanation of Docker Compose instructions:

Instruction	Purpose
services	Defines multiple containers
frontend	Creates the frontend service
ai-model	Creates the AI prediction service
database	Creates the PostgreSQL service
build	Builds an image using a Dockerfile
image	Uses an existing Docker image
ports	Maps host and container ports
environment	Sets environment variables
depends_on	Defines service startup dependencies
volumes	Preserves database data
healthcheck	Checks database readiness

The frontend communicates with the AI service using ai-model:5001. The AI service communicates with PostgreSQL using database. Docker Compose automatically creates a network for these services.

Step 7: Verify project structure

Run:

sudo apt install tree -y

Then:

tree

Expected output:

```
Docker-Compose-AI/
|
 ├── docker-compose.yml
|
 ├── frontend/
|    ├── app.py
|    ├── requirements.txt
|    └── Dockerfile
|
└── ai-model/
     ├── app.py
     ├── requirements.txt
     └── Dockerfile
```

Step 8: Build and start all containers

Make sure you are inside the Docker-Compose-AI directory.

Run:

sudo docker compose up -d --build

Explanation:

- docker compose manages multiple containers.
- up creates and starts the services.
- -d runs containers in the background.

- --build builds the images before starting them.

Wait until the build and startup process finishes.

Step 9: Verify all containers

Run:

sudo docker compose ps

Expected services:

Container	Service	Expected status
ai-frontend	Frontend	Running
ai-model	AI Model	Running
ai-database	Database	Running / Healthy

If any container is not running, check its logs:

sudo docker compose logs

To view only AI model logs:

sudo docker compose logs ai-model

Step 10: Test the web application

Open your browser and visit:

Open AI web application

You should see the AI Iris Flower Prediction page.

The default input values are:

Input	Value
Sepal Length	5.1
Sepal Width	3.5
Petal Length	1.4
Petal Width	0.2

Click the Predict button.

Expected result:

Prediction: setosa

The prediction should also appear in the Prediction History section because the AI service stores it in PostgreSQL.

Step 11: Test the AI API

Open another Ubuntu Terminal and execute:

curl http://localhost:5001/health

Expected output:

{"status":"AI service is running"}

Test the prediction API:

```
curl -X POST http://localhost:5001/predict \
-H "Content-Type: application/json" \
-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
```

Expected output:

```
{
  "message": "Prediction saved successfully",
  "prediction": "setosa"
}
```

Step 12: Verify database records

Run:

sudo docker compose exec database psql -U admin -d aidb

Inside the PostgreSQL prompt, execute:

SELECT * FROM predictions;

You should see the saved prediction records.

Exit PostgreSQL:

\q

Step 13: Stop all containers

After testing, execute:

sudo docker compose down

This stops and removes the containers and Compose network. The database volume remains, so saved prediction history is preserved.

Verify:

sudo docker ps

The three containers should no longer appear in the running-container list.

To stop the containers and also delete the saved database volume, use sudo docker compose down -v only when you intentionally want to remove the stored records.

5. Final project structure

Docker-Compose-AI/

|

├── docker-compose.yml

|

├── frontend/

| ├── app.py

| ├── requirements.txt

| └── Dockerfile

|

└── ai-model/

├── app.py

├── requirements.txt

└── Dockerfile

6. Important Docker Compose commands

Command	**Purpose**

docker compose version	Checks Compose version
docker compose build	Builds service images
docker compose up -d	Starts all services
docker compose up -d --build	Builds and starts all services
docker compose ps	Displays service status
docker compose logs	Displays service logs
docker compose stop	Stops services
docker compose down	Stops and removes containers and network
docker compose down -v	Also removes named volumes

7. Result

Successfully created a web-based AI application using Docker Compose with three separate containers for the frontend, AI model, and PostgreSQL database. Built and started the containers, tested AI predictions through the browser and API, verified prediction records in the database, and stopped all containers successfully.

8. Viva questions and answers

Q1. What is Docker Compose?

Docker Compose is a tool used to manage multiple Docker containers through a single YAML configuration file.

Q2. What is the use of docker-compose.yml?

It defines the services, images, ports, environment variables, and volumes required by an application.

Q3. What are the three services in this experiment?

Frontend, AI model, and database.

Q4. What is the purpose of the frontend container?

It provides the user interface for entering input and viewing predictions.

Q5. What is the purpose of the AI model container?

It performs machine-learning predictions using the trained model.

Q6. What is the purpose of the database container?

It stores prediction inputs and their results.

Q7. What is a Docker volume?

A Docker volume stores data outside the container so that it can persist even when the container is removed.

Q8. What is the purpose of docker compose up -d?

It creates and starts all defined services in the background.

Q9. What is the difference between Docker and Docker Compose?

Docker manages individual containers and images, while Docker Compose manages multiple related containers together.

Q10. What is port mapping?

Port mapping connects a host machine's port to a port inside a container.

Q11. Why do containers communicate using service names?

Docker Compose creates a shared network where services can discover and communicate with each other using their service names.

Q12. Why do we use PostgreSQL?

PostgreSQL is a relational database used to store and retrieve structured data.

Experiment 10 is complete.

Note: The database password in this classroom example is for demonstration only. A real application should use a strong password and keep credentials out of source code.

Experiment 11: Deploy and Scale a Dockerized AI Application Using Kubernetes and Minikube (Ubuntu)

1. Aim

To set up a local Kubernetes cluster using Minikube, deploy a Dockerized AI application, manage its frontend, AI model, and database services, and scale the application using Kubernetes.

2. Software requirements

- Ubuntu
- Docker
- Minikube
- kubectl
- Python and Flask
- Dockerized AI application from Experiment 10

3. Theory

Kubernetes: Kubernetes is an open-source container orchestration platform used to deploy, manage, and scale containerized applications.

Minikube: Minikube creates a local Kubernetes cluster on a computer. It is useful for learning and testing Kubernetes.

Pod: A Pod is the smallest deployable unit in Kubernetes. It runs one or more containers.

Deployment: A Deployment manages Pods and maintains the required number of application replicas.

Service: A Service provides a stable network endpoint to access Pods.

Scaling: Scaling means increasing or decreasing the number of application replicas.

kubectl: A command-line tool used to communicate with and manage Kubernetes clusters.

4. Architecture

We will use the same three-service AI application from Experiment 10.

User / Web Browser

localhost:5000

Kubernetes Cluster – Minikube

Frontend Deployment

Frontend Pods

AI Model Deployment

AI Model Pods

PostgreSQL Deployment

Database Pod + Persistent Volume

5. Procedure

We will deploy the same frontend, ai-model, and database services created in Experiment 10. If you have not created that project, use the same project files and Dockerfiles from Experiment 10 first.

5. Install and configure the required tools

Step 1: Open Ubuntu Terminal

Press Ctrl + Alt + T.

Check Docker:

docker --version

Check Minikube:

minikube version

Check kubectl:

kubectl version --client

If Docker is already installed in Ubuntu, continue to Step 2. Docker installed on Windows is not automatically available inside Ubuntu or WSL.

If Docker is not installed in Ubuntu, install Docker Engine using its official Ubuntu repository:

sudo apt update

sudo apt install ca-certificates curl -y

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \

-o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

Add the Docker repository:

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF

Types: deb

URIs: https://download.docker.com/linux/ubuntu

Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")

Components: stable

Architectures: $(dpkg --print-architecture)

Signed-By: /etc/apt/keyrings/docker.asc

EOF

Install Docker:

sudo apt update

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y

Start Docker:

sudo systemctl start docker

Verify:

sudo docker run hello-world

Docker installation instructions: Official Docker Ubuntu installation guide.

Docker Docs+1

Step 2: Install Minikube

Download the latest stable Minikube package for an x86-64 Ubuntu computer:

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb

Install it:

sudo dpkg -i minikube_latest_amd64.deb

Check the version:

minikube version

Expected output:

minikube version: ...

Official instructions: Minikube installation guide.

minikube

Step 3: Install kubectl

Download kubectl:

curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

Install:

sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

Verify:

kubectl version --client

Expected output:

Client Version: v...

Official instructions: Kubernetes kubectl installation guide.

Kubernetes

These download commands use the amd64 architecture. If uname -m returns aarch64, use the arm64 Minikube and kubectl downloads instead.

6. Start the local Kubernetes cluster

Step 4: Start Docker

sudo systemctl start docker

Check Docker:

sudo docker ps

If Docker commands work only with sudo, you can continue using sudo for Docker commands. Minikube's Docker driver may need access to the Docker daemon as your normal Ubuntu user. To enable that access:

sudo usermod -aG docker $USER

Then log out and log back in, or run:

newgrp docker

Membership in the Docker group grants powerful, effectively root-level access to the machine.

Docker Docs

Step 5: Start Minikube

Run:

minikube start --driver=docker --cpus=2 --memory=4096

This creates a local Kubernetes cluster using Docker.

Explanation:

- minikube start starts the cluster.
- --driver=docker uses Docker to run Minikube.
- --cpus=2 allocates two CPUs.
- --memory=4096 allocates 4 GB memory.

Wait until Minikube reports that the Kubernetes control plane is running.

The Docker driver is supported on Linux and does not require a separate hypervisor.

minikube+1

Step 6: Verify the Kubernetes cluster

kubectl cluster-info

Check the nodes:

kubectl get nodes

Expected output:

NAME STATUS ROLES AGE VERSION

minikube Ready control-plane ... ...

Expected status: Ready

Check system Pods:

kubectl get pods -A

The cluster is ready when the Minikube node reports Ready.

7. Prepare the Dockerized AI application

Step 7: Open your Experiment 10 project

If you already created the project in Experiment 10, open its directory:

cd ~/Docker-Compose-AI

Check the files:

ls

You should see:

ai-model

frontend

docker-compose.yml

The ai-model and frontend folders must contain the Python files, requirements files, and Dockerfiles created in Experiment 10.

Create a folder for Kubernetes configuration:

mkdir -p k8s

Step 8: Build Docker images inside Minikube

We need to make the frontend and AI model images available to the Minikube cluster.

Build the AI model image:

minikube image build -t ai-model:latest ./ai-model

Build the frontend image:

minikube image build -t ai-frontend:latest ./frontend

Check the images:

minikube image ls

You should find:

ai-model:latest

ai-frontend:latest

Important: We use minikube image build so that the images are available inside the Minikube cluster. We will set imagePullPolicy: Never in the Kubernetes configuration so Kubernetes uses these local images.

8. Create Kubernetes configuration files

We will create two YAML files:

- database.yaml – PostgreSQL database deployment and service.
- app.yaml – AI model and frontend deployments and services.

Step 9: Create the database configuration

Run:

nano k8s/database.yaml

Paste the following code:

apiVersion: v1

kind: Secret

metadata:

  name: db-secret

type: Opaque

stringData:

  POSTGRES_DB: aidb

  POSTGRES_USER: admin

  POSTGRES_PASSWORD: admin123

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi


---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: database
spec:
  replicas: 1
  selector:
    matchLabels:
      app: database
  template:
    metadata:
      labels:
        app: database
    spec:
```

```
containers:
  - name: postgres
    image: postgres:16
    ports:
      - containerPort: 5432
    env:
      - name: POSTGRES_DB
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: POSTGRES_DB

      - name: POSTGRES_USER
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: POSTGRES_USER

      - name: POSTGRES_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: POSTGRES_PASSWORD

    volumeMounts:
      - name: postgres-storage
```

```
            mountPath: /var/lib/postgresql/data

        readinessProbe:
          exec:
            command:
              - pg_isready
              - -U
              - admin
              - -d
              - aidb
          initialDelaySeconds: 10
          periodSeconds: 5

      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc

---
apiVersion: v1
kind: Service
metadata:
  name: database
spec:
  selector:
    app: database
```

```
  ports:
    - port: 5432
      targetPort: 5432
  type: ClusterIP
```

Save the file using Ctrl + O, Enter, and Ctrl + X.

Explanation:

- Secret stores the database credentials.
- PersistentVolumeClaim requests storage for PostgreSQL.
- Deployment creates the database Pod.
- replicas: 1 runs one database replica.
- Service allows other Kubernetes Pods to connect to the database.
- readinessProbe checks whether PostgreSQL is ready.

Step 10: Create AI model and frontend configurations

Run:

```
nano k8s/app.yaml
```

Paste:

```
# AI MODEL DEPLOYMENT


apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-model
spec:
  replicas: 1
  selector:
    matchLabels:
```

```
    app: ai-model
  template:
    metadata:
      labels:
        app: ai-model
    spec:
      containers:
        - name: ai-model
          image: ai-model:latest
          imagePullPolicy: Never

          ports:
            - containerPort: 5001

          env:
            - name: DB_HOST
              value: database

            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: POSTGRES_DB

            - name: POSTGRES_USER
              valueFrom:
```

```
            secretKeyRef:
              name: db-secret
              key: POSTGRES_USER

          - name: POSTGRES_PASSWORD
            valueFrom:
              secretKeyRef:
                name: db-secret
                key: POSTGRES_PASSWORD

        readinessProbe:
          httpGet:
            path: /health
            port: 5001
          initialDelaySeconds: 15
          periodSeconds: 5

        livenessProbe:
          httpGet:
            path: /health
            port: 5001
          initialDelaySeconds: 30
          periodSeconds: 10

---
# AI MODEL SERVICE
```

```
apiVersion: v1
kind: Service
metadata:
  name: ai-model
spec:
  selector:
    app: ai-model
  ports:
    - port: 5001
      targetPort: 5001
  type: ClusterIP

---
# FRONTEND DEPLOYMENT

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
```

```
metadata:
  labels:
    app: frontend
spec:
  containers:
    - name: frontend
      image: ai-frontend:latest
      imagePullPolicy: Never

      ports:
        - containerPort: 5000

      env:
        - name: AI_API_URL
          value: http://ai-model:5001

      readinessProbe:
        httpGet:
          path: /health
          port: 5000
        initialDelaySeconds: 10
        periodSeconds: 5

      livenessProbe:
        httpGet:
          path: /health
```

```
        port: 5000
      initialDelaySeconds: 20
      periodSeconds: 10


---
# FRONTEND SERVICE


apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30080
  type: NodePort
```

Save the file.

Explanation:

Kubernetes object	**Purpose**
Deployment	Creates and manages application Pods
Pod	Runs the AI or frontend container
Service	Provides network communication

ClusterIP	Allows internal cluster communication
NodePort	Makes the frontend accessible through a node port
Secret	Supplies database credentials
PersistentVolumeClaim	Requests persistent database storage
Replicas	Specifies how many Pods should run

The frontend communicates with the AI model using http://ai-model:5001. The AI model connects to PostgreSQL using the service name database.

9. Deploy the application

Step 11: Deploy the database

Run:

kubectl apply -f k8s/database.yaml

Expected output:

secret/db-secret created

persistentvolumeclaim/postgres-pvc created

deployment.apps/database created

service/database created

Wait until the database is ready:

kubectl rollout status deployment/database

Expected output:

deployment "database" successfully rolled out

Step 12: Deploy the AI model and frontend

Run:

kubectl apply -f k8s/app.yaml

Expected output:

deployment.apps/ai-model created

service/ai-model created

deployment.apps/frontend created

service/frontend created

Wait for the AI model:

kubectl rollout status deployment/ai-model

Wait for the frontend:

kubectl rollout status deployment/frontend

Both should report successful rollout.

Step 13: Verify Pods

Run:

kubectl get pods

Expected output will contain three Pods:

NAME READY STATUS

database-xxxxxxxxxx-xxxxx 1/1 Running

ai-model-xxxxxxxxxx-xxxxx 1/1 Running

frontend-xxxxxxxxxx-xxxxx 1/1 Running

Expected status: Running

Check deployments:

kubectl get deployments

Check services:

kubectl get services

You should find:

- database
- ai-model
- frontend

10. Access and test the AI application

Step 14: Open the frontend service

Run:

minikube service frontend --url

Minikube will display a URL similar to:

http://192.168.49.2:30080

Open the displayed URL in your browser.

You should see:

AI Iris Flower Prediction

Example of the frontend application

Sepal Length

5.1

Sepal Width

3.5

Petal Length

1.4

Petal Width

0.2

Predict

Expected prediction: setosa

Enter the four values and click Predict.

Expected result:

Prediction: setosa

The application also displays prediction history retrieved from PostgreSQL.

Alternative if the Minikube URL does not open: Run this command in a separate Ubuntu Terminal:

kubectl port-forward service/frontend 5000:5000

Keep that terminal open and visit:

Open the application on localhost:5000.

Step 15: Test the AI model API

Open another Ubuntu Terminal.

Check the AI service:

kubectl port-forward service/ai-model 5001:5001

Keep this terminal open.

In another terminal, execute:

curl http://localhost:5001/health

Expected output:

{"status":"AI service is running"}

Test the prediction API:

curl -X POST http://localhost:5001/predict \

-H "Content-Type: application/json" \

-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'

Expected output:

{

"message": "Prediction saved successfully",

"prediction": "setosa"

}

Step 16: Verify database records

Run:

kubectl exec deployment/database -- \

psql -U admin -d aidb -c "SELECT * FROM predictions;"

You should see the prediction records stored in PostgreSQL.

11. Scale the application using Kubernetes

Step 17: Increase frontend replicas

Initially, the frontend deployment has one replica.

Check:

kubectl get deployments

Increase frontend replicas to three:

kubectl scale deployment frontend --replicas=3

Verify:

kubectl get deployments

Expected frontend status:

NAME READY UP-TO-DATE AVAILABLE

frontend 3/3 3 3

Step 18: Scale the AI model

Increase AI model replicas to two:

kubectl scale deployment ai-model --replicas=2

Verify:

kubectl get deployments

Expected output:

NAME READY UP-TO-DATE AVAILABLE

ai-model 2/2 2 2

The database remains at one replica because this example uses a single PostgreSQL database with persistent storage.

Step 19: Verify all running Pods

Run:

kubectl get pods

You should now have:

Service	**Number of Pods**
Frontend	3

AI Model 2

Database 1

Total 6

Scaling successful

Kubernetes maintains the requested number of replicas through Deployments.

Step 20: Scale down the application

To return to one frontend and one AI model replica:

kubectl scale deployment frontend --replicas=1

kubectl scale deployment ai-model --replicas=1

Verify:

kubectl get deployments

12. Stop the Kubernetes cluster

After completing the experiment, stop Minikube:

minikube stop

Expected output:

Done! kubectl is now configured to use "minikube" cluster

The cluster is stopped, but its configuration and deployed resources are retained. To start it again:

minikube start

13. Final project structure

Docker-Compose-AI/

|

├— docker-compose.yml

|

├— frontend/

| ├— app.py

```
|   ├── requirements.txt
|   └── Dockerfile
|
├── ai-model/
|   ├── app.py
|   ├── requirements.txt
|   └── Dockerfile
|
└── k8s/
    ├── database.yaml
    └── app.yaml
```

14. Important Kubernetes commands

Command	Purpose
minikube start	Starts local cluster
minikube stop	Stops local cluster
minikube status	Checks cluster status
kubectl get nodes	Displays cluster nodes
kubectl get pods	Displays Pods
kubectl get deployments	Displays deployments
kubectl get services	Displays services
kubectl apply -f file.yaml	Creates resources
kubectl scale deployment frontend --replicas=3	Scales frontend
kubectl describe pod POD_NAME	Displays Pod details
kubectl logs POD_NAME	Displays Pod logs

kubectl delete -f file.yaml	Deletes resources defined in a file

15. Result

Successfully installed and configured Minikube, created a local Kubernetes cluster, deployed a Dockerized AI application consisting of frontend, AI model, and PostgreSQL database services, tested the application, and scaled the frontend and AI model using Kubernetes Deployments.

16. Viva questions and answers

Q1. What is Kubernetes?

Kubernetes is an open-source platform used to deploy, manage, and scale containerized applications.

Q2. What is Minikube?

Minikube is a tool used to run a local Kubernetes cluster on a computer.

Q3. What is kubectl?

kubectl is a command-line tool used to communicate with a Kubernetes cluster.

Q4. What is a Pod?

A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers.

Q5. What is a Deployment?

A Deployment manages Pods and maintains the desired number of replicas.

Q6. What is a Kubernetes Service?

A Service provides a stable network endpoint for accessing application Pods.

Q7. What is scaling?

Scaling means increasing or decreasing the number of application replicas.

Q8. What is Minikube's Docker driver?

It allows Minikube to use Docker as the container environment for its local Kubernetes cluster.

Q9. What is a YAML file?

YAML is a human-readable data format used to configure Kubernetes resources.

Q10. What is the purpose of kubectl apply?

It creates or updates Kubernetes resources using configuration files.

Q11. What is NodePort?

NodePort exposes a Kubernetes Service through a port on the cluster node.

Q12. What is the difference between Docker and Kubernetes?

Docker is used to build and run containers, whereas Kubernetes manages and orchestrates containers across a cluster.

Q13. What is a PersistentVolumeClaim?

It is a request for storage used by an application in Kubernetes.

Q14. What is the purpose of replicas?

Replicas maintain multiple running copies of an application for availability and scaling.

Important notes:

- This practical reuses the frontend and AI model code from Experiment 10.
- Build the images inside Minikube before deploying them.
- Do not scale the PostgreSQL Deployment like a stateless frontend; a production database needs a proper StatefulSet and replication strategy.
- The database password is only for this classroom demonstration.

Official references:

- Minikube documentation
- Kubernetes documentation
- Minikube Docker driver.

Experiment 12: CI/CD Pipeline Using Jenkins, Docker, and Kubernetes for an AI Model (Ubuntu)

1. Aim

To create a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins, Docker, and Kubernetes to automate the process of testing, building, and deploying a Dockerized AI application.

2. Software requirements

- Ubuntu
- Jenkins
- Docker
- Kubernetes
- Minikube
- kubectl
- Git and GitHub
- Docker Hub account
- AI application from Experiment 10

3. Theory

CI (Continuous Integration): Automatically checks and tests code whenever developers push changes to GitHub.

CD (Continuous Deployment): Automatically deploys the tested application to a server or Kubernetes cluster.

Jenkins: An automation server used to create and execute CI/CD pipelines.

Docker: Packages an application and its dependencies into a Docker image.

Kubernetes: Deploys, manages, and scales containerized applications.

4. Working of the CI/CD pipeline

1

Developer

Changes AI application code

2

GitHub

Stores and receives updated code

3

Jenkins

Automatically starts the pipeline

4

Testing

Checks application code

5

Docker

Builds and pushes Docker images

6

Kubernetes

Deploys the updated AI application

7

Verification

Checks application and deployment status

Application used: We will reuse the three-service AI application from Experiment 10: frontend, AI model, and PostgreSQL database. Kubernetes configuration from Experiment 11 will be reused.

5. Check the required tools in Ubuntu

Open Ubuntu Terminal using Ctrl + Alt + T.

Run these commands one by one:

git --version

docker --version

docker compose version

java -version

jenkins --version

minikube version

kubectl version --client

Important: You previously mentioned Jenkins, Docker, and Kubernetes are installed on Windows. That does not necessarily mean they are installed in Ubuntu. This practical must run its Jenkins, Docker, and Kubernetes commands in the Ubuntu environment.

If a command says command not found, that tool needs to be configured in Ubuntu before proceeding.

6. Install Jenkins in Ubuntu (if not already installed)

If jenkins --version worked in Step 5, skip the installation commands and proceed to Step 7.

Step 1: Install Java

Jenkins requires a supported Java version. The current Jenkins Linux installation guide uses Java 21.

Jenkins+1

sudo apt update

sudo apt install fontconfig openjdk-21-jre -y

Verify:

java -version

Step 2: Add the Jenkins repository

sudo mkdir -p /etc/apt/keyrings

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \

https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

Add the repository:

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | \

sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

Install Jenkins:

sudo apt update

sudo apt install jenkins -y

Step 3: Start Jenkins

sudo systemctl start jenkins

Enable Jenkins at startup:

sudo systemctl enable jenkins

Check status:

sudo systemctl status jenkins

Expected:

Active: active (running)

Step 4: Open Jenkins

Open your browser:

Open Jenkins dashboard

Get the initial administrator password:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Copy the password and paste it into the Jenkins setup page.

Then:

1. Click Install suggested plugins.
2. Wait for plugin installation.
3. Create your administrator account.
4. Open the Jenkins dashboard.

Official installation guide: Jenkins installation for Linux.

Jenkins

7. Configure Jenkins plugins and permissions

Step 5: Install required plugins

In Jenkins:

1. Open Manage Jenkins.
2. Select Plugins.
3. Open Available plugins.
4. Search for and install:

Plugin	Purpose
Git	Connects Jenkins to GitHub
Pipeline	Creates CI/CD pipelines
Credentials Binding	Uses stored credentials securely

Restart Jenkins if requested.

We will use Docker and kubectl command-line tools directly in our Jenkinsfile, so the Docker Pipeline plugin is not required for this particular implementation.

Step 6: Give Jenkins permission to use Docker

In Ubuntu Terminal:

sudo usermod -aG docker jenkins

Restart Jenkins:

sudo systemctl restart jenkins

Check Docker access for the Jenkins user:

sudo -u jenkins docker version

You should see Docker client and server version information.

If you receive a permission error, check the Docker socket permissions and Jenkins group membership before proceeding.

8. Create a GitHub repository

Step 7: Create the repository

Open GitHub:

Create a new GitHub repository

Create a repository with the name:

AI-CICD-Kubernetes

Choose Public for this classroom demonstration.

Do not initialize it with a README if you plan to push your existing project directly.

Step 8: Prepare the project

Open the Experiment 10 project:

cd ~/Docker-Compose-AI

Check:

ls

Your project should contain:

ai-model/

frontend/

docker-compose.yml

k8s/

The k8s folder contains the Kubernetes configuration files from Experiment 11.

Create or update the files as described below.

Step 9: Update Kubernetes image names

Open:

nano k8s/app.yaml

In the AI model Deployment, use this image configuration:

image: DOCKERHUB_USERNAME/ai-model:IMAGE_TAG

imagePullPolicy: Always

In the frontend Deployment, use:

image: DOCKERHUB_USERNAME/ai-frontend:IMAGE_TAG

imagePullPolicy: Always

Keep the remaining Deployment and Service configurations from Experiment 11.

The placeholders will be replaced automatically by Jenkins with your Docker Hub username and build number.

Database configuration: Keep the k8s/database.yaml file from Experiment 11. It defines the PostgreSQL Deployment, Secret, persistent storage, and database Service.

9. Create Docker Hub repositories

Step 10: Create Docker Hub account and repositories

Open:

Docker Hub

Sign in or create an account.

Create these two repositories:

ai-model

ai-frontend

For this practical, make both repositories public so that Minikube can pull the images without additional registry authentication.

You will need your Docker Hub username and an access token.

To create an access token:

1. Open Docker Hub account settings.
2. Find Personal access tokens.
3. Create a new token with read and write permissions.
4. Copy the token and keep it private.

10. Create the Jenkinsfile

Step 11: Create Jenkinsfile in the project directory

Return to Ubuntu Terminal:

cd ~/Docker-Compose-AI

Create:

nano Jenkinsfile

Paste this complete pipeline:

```
pipeline {

    agent any

    triggers {
        pollSCM('H/2 * * * *')
    }

    environment {
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    stages {

        stage('Checkout') {
            steps {
                echo 'Getting project source code from GitHub'
                checkout scm
            }
        }

        stage('Test') {
            steps {
```

```
        echo 'Checking Python application files'

        sh '''
            python3 -m py_compile ai-model/app.py
            python3 -m py_compile frontend/app.py
        '''

        echo 'Python code test successful'
    }
}

stage('Build Docker Images') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKERHUB_USER',
                passwordVariable: 'DOCKERHUB_TOKEN'
            )
        ]) {
            sh '''
                docker build \
                -t $DOCKERHUB_USER/ai-model:$IMAGE_TAG \
                -t $DOCKERHUB_USER/ai-model:latest \
                ./ai-model
```

```
            docker build \
            -t $DOCKERHUB_USER/ai-frontend:$IMAGE_TAG \
            -t $DOCKERHUB_USER/ai-frontend:latest \
            ./frontend
        '''
      }
    }
}

stage('Push Docker Images') {
    steps {
      withCredentials([
        usernamePassword(
          credentialsId: 'dockerhub-creds',
          usernameVariable: 'DOCKERHUB_USER',
          passwordVariable: 'DOCKERHUB_TOKEN'
        )
      ]) {
        sh '''
          echo "$DOCKERHUB_TOKEN" | \
          docker login -u "$DOCKERHUB_USER" \
          --password-stdin

          docker push $DOCKERHUB_USER/ai-model:$IMAGE_TAG
          docker push $DOCKERHUB_USER/ai-model:latest
```

```
            docker push $DOCKERHUB_USER/ai-frontend:$IMAGE_TAG
            docker push $DOCKERHUB_USER/ai-frontend:latest

            docker logout
          '''
        }
      }
}

stage('Deploy to Kubernetes') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKERHUB_USER',
                passwordVariable: 'DOCKERHUB_TOKEN'
            ),
            file(
                credentialsId: 'minikube-kubeconfig',
                variable: 'KUBECONFIG'
            )
        ]) {
            sh '''
                kubectl apply -f k8s/database.yaml

                kubectl rollout status deployment/database \
```

```
                --timeout=180s

                sed \
                -e "s|DOCKERHUB_USERNAME|$DOCKERHUB_USER|g" \
                -e "s|IMAGE_TAG|$IMAGE_TAG|g" \
                k8s/app.yaml > app-deploy.yaml

                kubectl apply -f app-deploy.yaml

                kubectl rollout status deployment/ai-model \
                --timeout=300s

                kubectl rollout status deployment/frontend \
                --timeout=300s

                kubectl get pods
                kubectl get services
            '''
        }
      }
    }

}

post {
    success {
```

```
            echo 'CI/CD Pipeline completed successfully!'
        }

        failure {
            echo 'CI/CD Pipeline failed. Check the console output.'
        }
    }
}
```

Save the file.

Explanation of pipeline stages:

Stage	**Work performed**
Checkout	Downloads source code from GitHub
Test	Checks Python code syntax
Build Docker Images	Creates frontend and AI images
Push Docker Images	Uploads images to Docker Hub
Deploy to Kubernetes	Updates application in Minikube
Post	Displays success or failure message

The pollSCM trigger checks GitHub approximately every two minutes for new changes. Jenkins can then start another build when it detects a change.

11. Push the project to GitHub

Step 12: Initialize Git

Run:

```
git init
```

Set the main branch:

```
git branch -M main
```

Add your GitHub repository:

git remote add origin https://github.com/YOUR_USERNAME/AI-CICD-Kubernetes.git

Replace YOUR_USERNAME with your actual GitHub username.

Add files:

git add .

Commit:

git commit -m "Add AI CI CD project with Jenkins Docker Kubernetes"

Push:

git push -u origin main

If GitHub asks for a password, use a Personal Access Token or another supported GitHub authentication method instead of your account password.

Check GitHub and verify that the project files and Jenkinsfile have been uploaded.

12. Configure Jenkins credentials

Jenkins needs two credentials:

1. Docker Hub username and access token.
2. Kubernetes configuration file for Minikube.

Step 13: Create Docker Hub credentials

First, open Jenkins:

Open Jenkins dashboard

Follow these steps:

1. Click Manage Jenkins.
2. Click Credentials.
3. Open System.
4. Open Global credentials (unrestricted).
5. Click Add Credentials.

Enter:

Field	Value
Kind	Username with password
Username	Your Docker Hub username
Password	Your Docker Hub access token
ID	dockerhub-creds
Description	Docker Hub credentials

Click Create.

Step 14: Create Minikube kubeconfig file

First, make sure Minikube is running:

minikube status

Check cluster access:

kubectl get nodes

Create a standalone Kubernetes configuration file:

kubectl config view --raw --flatten --minify > ~/minikube-kubeconfig

Check that the file exists:

ls -l ~/minikube-kubeconfig

Now upload it into Jenkins:

1. Open Manage Jenkins.
2. Open Credentials.
3. Select System.
4. Select Global credentials (unrestricted).
5. Click Add Credentials.

Enter:

Field	Value

Kind	Secret file
File	Select minikube-kubeconfig
ID	minikube-kubeconfig
Description	Minikube Kubernetes configuration

Click Create.

This allows the Jenkins pipeline to access your Minikube cluster without putting Kubernetes credentials directly inside the Jenkinsfile.

Keep the kubeconfig file private. It contains credentials that grant access to your cluster.

13. Create the Jenkins Pipeline job

Step 15: Create a new Pipeline

Open the Jenkins dashboard.

1. Click New Item.
2. Enter the name:

AI-CICD-Pipeline

3. Select Pipeline.
4. Click OK.

Step 16: Configure the GitHub repository

Scroll to the Pipeline section.

Select:

Definition: Pipeline script from SCM

Enter:

Field	**Value**
SCM	Git
Repository URL	https://github.com/YOUR_USERNAME/AI-CICD-Kubernetes.git
Credentials	None for a public repository

Branch */main

Script Path Jenkinsfile

Replace YOUR_USERNAME with your GitHub username.

Click Save.

Step 17: Configure automatic triggering

The Jenkinsfile already contains:

```
triggers {
   pollSCM('H/2 * * * *')
}
```

This checks the GitHub repository approximately every two minutes.

No additional webhook setup is required for this experiment.

14. Execute the CI/CD pipeline

Step 18: Start the pipeline

Open:

AI-CICD-Pipeline

Click:

Build Now

Jenkins will start executing the pipeline stages.

Expected pipeline stages:

Checkout

Test

Build Docker Images

Push Docker Images

Deploy to Kubernetes

These are the expected successful stages, not a report of an actual build.

Step 19: Check Jenkins Console Output

1. Open the latest build number.
2. Click Console Output.
3. Observe each stage.

Expected final message:

CI/CD Pipeline completed successfully!

Finished: SUCCESS

If the pipeline fails, inspect the Console Output to identify the failing stage.

15. Verify Kubernetes deployment

Step 20: Check running Pods

Open Ubuntu Terminal:

kubectl get pods

Expected output:

NAME READY STATUS

database-xxxxxxxxxx-xxxxx 1/1 Running

ai-model-xxxxxxxxxx-xxxxx 1/1 Running

frontend-xxxxxxxxxx-xxxxx 1/1 Running

Step 21: Check deployments

kubectl get deployments

Expected:

NAME READY UP-TO-DATE AVAILABLE

database 1/1 1 1

ai-model 1/1 1 1

frontend 1/1 1 1

Step 22: Check services

kubectl get services

You should see:

NAME TYPE PORT(S)

database ClusterIP 5432/TCP

ai-model ClusterIP 5001/TCP

frontend NodePort 5000:30080/TCP

Step 23: Test the deployed AI application

Run:

minikube service frontend --url

Copy the URL displayed in your terminal and open it in your browser.

You should see the AI Iris Flower Prediction page.

Enter:

Input	Value
Sepal Length	5.1
Sepal Width	3.5
Petal Length	1.4
Petal Width	0.2

Click Predict.

Expected output:

Prediction: setosa

The prediction history should also display the saved prediction.

16. Test automatic CI/CD triggering

Step 24: Make a change to the application

Open:

nano frontend/app.py

Add a harmless comment near the beginning:

Updated through Jenkins CI/CD pipeline

Save the file.

Push the changes:

git add .

git commit -m "Update AI frontend application"

git push

Wait approximately two minutes for Jenkins to poll GitHub.

Open the Jenkins dashboard and check whether a new build has started.

Step 25: Verify the new deployment

After the pipeline succeeds, run:

kubectl get pods

Check the latest deployment:

kubectl rollout status deployment/frontend

Check the latest application logs:

kubectl logs deployment/frontend

The updated Docker image should now be deployed to Kubernetes.

17. Troubleshooting

Problem	**Solution**
Jenkins is not opening	Run sudo systemctl status jenkins
Docker permission denied	Add Jenkins to Docker group and restart Jenkins
Jenkins cannot find kubectl	Check which kubectl and Jenkins PATH
Kubernetes connection refused	Start Minikube and verify kubeconfig
ImagePullBackOff	Check Docker Hub username, repository name, and image tag
Pod is not running	Run kubectl describe pod POD_NAME
Pipeline failed	Check Jenkins Console Output

Database connection failed	Check database Pod and logs

18. Important commands

Command	Purpose
sudo systemctl start jenkins	Starts Jenkins
sudo systemctl status jenkins	Checks Jenkins status
docker build	Builds Docker image
docker push	Pushes image to Docker Hub
kubectl get pods	Displays running Pods
kubectl get deployments	Displays deployments
kubectl get services	Displays services
kubectl apply -f file.yaml	Applies Kubernetes configuration
kubectl rollout status deployment/frontend	Checks deployment status
minikube status	Checks Minikube cluster

19. Result

Successfully configured a CI/CD pipeline using Jenkins, Docker, and Kubernetes for an AI application. Integrated GitHub with Jenkins, automated code testing, Docker image building and pushing, and Kubernetes deployment. Verified the application and tested automatic pipeline triggering after pushing code changes to GitHub.

20. Viva questions and answers

Q1. What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Deployment. It automates the process of testing, building, and deploying applications.

Q2. What is Jenkins?

Jenkins is an open-source automation server used to create CI/CD pipelines.

Q3. What is a Jenkinsfile?

A Jenkinsfile is a text file containing instructions for executing a Jenkins pipeline.

Q4. What is Docker used for in CI/CD?

Docker packages applications and their dependencies into images.

Q5. What is Kubernetes used for?

Kubernetes manages, deploys, and scales containerized applications.

Q6. What is Docker Hub?

Docker Hub is an online registry used to store and share Docker images.

Q7. What is a pipeline stage?

A stage is a separate section of work in a Jenkins pipeline, such as testing, building, or deploying.

Q8. What is Continuous Integration?

Continuous Integration automatically integrates and tests code changes from developers.

Q9. What is Continuous Deployment?

Continuous Deployment automatically releases tested application changes to the deployment environment.

Q10. What is the purpose of GitHub in CI/CD?

GitHub stores source code and provides a repository from which Jenkins retrieves updates.

Q11. What is Minikube?

Minikube is a tool used to run a local Kubernetes cluster.

Q12. What is the purpose of Docker image tagging?

Image tags identify different versions of Docker images.

Q13. What is the purpose of kubectl apply?

It creates or updates Kubernetes resources using YAML configuration files.

Q14. What is the advantage of CI/CD?

It reduces manual work, improves deployment speed, and helps detect errors early.

Official references

- Jenkins Pipeline documentation
- Jenkinsfile documentation

- Docker documentation
- Kubernetes documentation
- Minikube documentation

Important: This practical uses Docker Hub to transfer images from Jenkins to Minikube. Keep the two Docker Hub repositories public for this basic setup, and use Jenkins credentials rather than writing access tokens in your Jenkinsfile.

Experiment 12 complete